

PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR 2025-2026 | FIFTH SEMESTER B.TECH

SOFTWARE ARCHITECTURE AND DESIGN

(SAD Specification)

PROJECT: SPRINTFLOW – PROJECT MANAGEMENT SYSTEM

(Workflow, Visibility, Estimation, and Kanban)

SUBMITTED BY: TEAM 11 (SECTION 5C)

Student Name	USN	Role	Assigned Deliverable
Darshan P Pawar	PES2UG24CS143	Team Lead & Architect	Architecture, ADRs, Component Diagram
Dhatri Shivaprasad	PES2UG24CS158	Frontend & UI/UX Lead	Design Overview, UX Design, Sequence Diagrams
Chandan Kumar K	PES2UG24CS128	Backend & Database Architect	API Design, Data Stores, Security Architecture
Gurubelli Yekambar Eshwar Rao	PES2UG24CS173	Full-Stack & QA/DevOps Lead	Test Strategy, Metrics, CI/CD Pipeline

Version: 1.0 (Final Submission) | Date: 15-09-2026 | Status: Approved

Repository: <https://github.com/Darshanpawar7/SE-SRS-2>

Version: 1.0

Authors: Team 11 (Section 5C, PES University)

Date: 15-09-2026

Status: Final Submission / Approved

Revision History

Version	Date	Author	Change Summary
1.0	15-09-2026	Team 11	Complete SAD with UML Component Diagram, 2 Sequence Diagrams, API Design, STRIDE Threat Model, and Architecture Traceability

Approvals

Role	Name	Department / Contact	Sign / Date
Team Lead	Darshan P Pawar	PES2UG24CS143 (CSE)	Darshan P Pawar / 15-09-2026
Course Coordinator	Software Engineering Faculty	Dept of CSE, PES University	_____ / _____

1. Introduction

1.1 Purpose

This Software Architecture and Design (SAD) document specifies the architecture, component structure, design patterns, API contracts, and security architecture of SprintFlow, an enterprise-grade Agile Project Management System. It serves as the authoritative technical reference for developers, quality assurance engineers, and assessment evaluators, providing complete traceability from architectural decisions to the functional and non-functional requirements defined in the Software Requirements Specification (SRS v1.0).

1.2 Scope

This document covers the complete software architecture and detailed design of SprintFlow, including:

- Component-based layered architecture of the React 18 single-page application
- Dual-mode data persistence layer (Supabase PostgreSQL cloud sync and browser LocalStorage offline fallback)
- UML component diagram illustrating all system modules and their inter-dependencies
- UML sequence diagrams modeling two core application workflows
- RESTful API design for Supabase backend integration
- STRIDE-based threat modeling and security architecture
- Technology stack rationale and architectural decision records

Out of scope: Supabase internal infrastructure implementation, browser rendering engine internals, and third-party CDN operations.

1.3 Audience

- **Developers:** Reference for implementing and extending SprintFlow components
- **QA Engineers:** Basis for deriving integration and system test cases from architectural contracts
- **Security Auditors:** STRIDE threat model and security control documentation
- **Course Evaluators:** Assessment of architectural design quality and IEEE 42010 compliance
- **Maintenance Engineers:** Component dependency map for impact analysis during future modifications

1.4 Definitions

Term	Definition
SAD	Software Architecture and Design Specification
SRS	Software Requirements Specification
STP	Software Test Plan
SPA	Single Page Application
RLS	Row Level Security (PostgreSQL policy-based access control)
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
ADR	Architecture Decision Record
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
RTM	Requirements Traceability Matrix
TLS	Transport Layer Security
XSS	Cross-Site Scripting
RBAC	Role-Based Access Control
KPI	Key Performance Indicator

2. Document Overview

2.1 How to Use This Document

This document is organized into three primary sections:

- **Section 3 (Architecture):** Provides the high-level system decomposition, component relationships, technology stack decisions, and security architecture. Start here for understanding the overall system structure.
- **Section 4 (Design):** Contains detailed design specifications including sequence diagrams for core workflows, API contracts, error handling strategies, and UX design principles. Reference this section for implementation-level guidance.
- **Section 5 (Appendices):** Contains the glossary, normative references, and tooling documentation.

All UML diagrams conform to UML 2.5 notation standards. Architecture decisions follow the Architecture Decision Record (ADR) format where applicable.

2.2 Related Documents

Document	Version	Repository
Software Requirements Specification (SRS)	1.0	[SE-SRS](https://github.com/Darshanpawar7/SE-SRS)
Software Test Plan (STP)	1.0	[SE-SRS-2](https://github.com/Darshanpawar7/SE-SRS-2)
Requirements Traceability Matrix (RTM)	1.0	Included in SRS Section 8
Database Schema (DDL)	1.0	database/schema.sql
CI/CD Workflow Definition	1.0	.github/workflows/ci.yml

3. Architecture

3.1 Goals and Constraints

Architectural Goals:

- **Modularity:** Independent development and testing of each functional module (Kanban, Dashboard, Sprint Manager, Estimation, Settings)
- **Resilience:** Continuous operation regardless of network connectivity through dual-mode data persistence
- **Security:** Defense-in-depth strategy with Row Level Security, input sanitization, and encrypted transport
- **Performance:** Sub-50ms UI state transitions for drag-and-drop operations and real-time dashboard updates

- **Maintainability:** Clear component boundaries enabling independent modification without cascading side effects
- **Availability:** Target 99.9% uptime through stateless client architecture and resilient data fallback

Architectural Constraints:

- Must operate as a client-side SPA without requiring server-side rendering infrastructure
- Must support zero-configuration deployment for academic evaluation (no mandatory database setup)
- Must integrate with Supabase PostgreSQL using the @supabase/supabase-js v2 client library
- Must pass automated CI/CD validation (build, test, lint) on every push and pull request via GitHub Actions
- Technology stack must align with the course-approved options: JavaScript (MERN Stack) or equivalent

3.2 Stakeholders and Concerns

Stakeholder	Primary Concerns
Team Lead (Darshan)	System reliability, architecture coherence, deployment simplicity, team coordination
Frontend Developer (Dhatri)	UI responsiveness, component reusability, drag-and-drop performance, visual consistency
Backend Developer (Chandan)	Data integrity, schema design, RLS policy correctness, query performance
DevOps/QA Lead (Eshwar)	Test coverage, CI/CD pipeline reliability, build reproducibility, estimation accuracy
Course Evaluators	IEEE compliance, documentation quality, functional completeness, collaboration evidence
End Users (Students)	Intuitive workflow management, real-time visibility, estimation accuracy

3.3 Component (UML) Diagram

The following UML Component Diagram illustrates the high-level decomposition of SprintFlow into its constituent modules, their provided/required interfaces, and inter-component dependencies:

UML Component Diagram: SprintFlow System Architecture

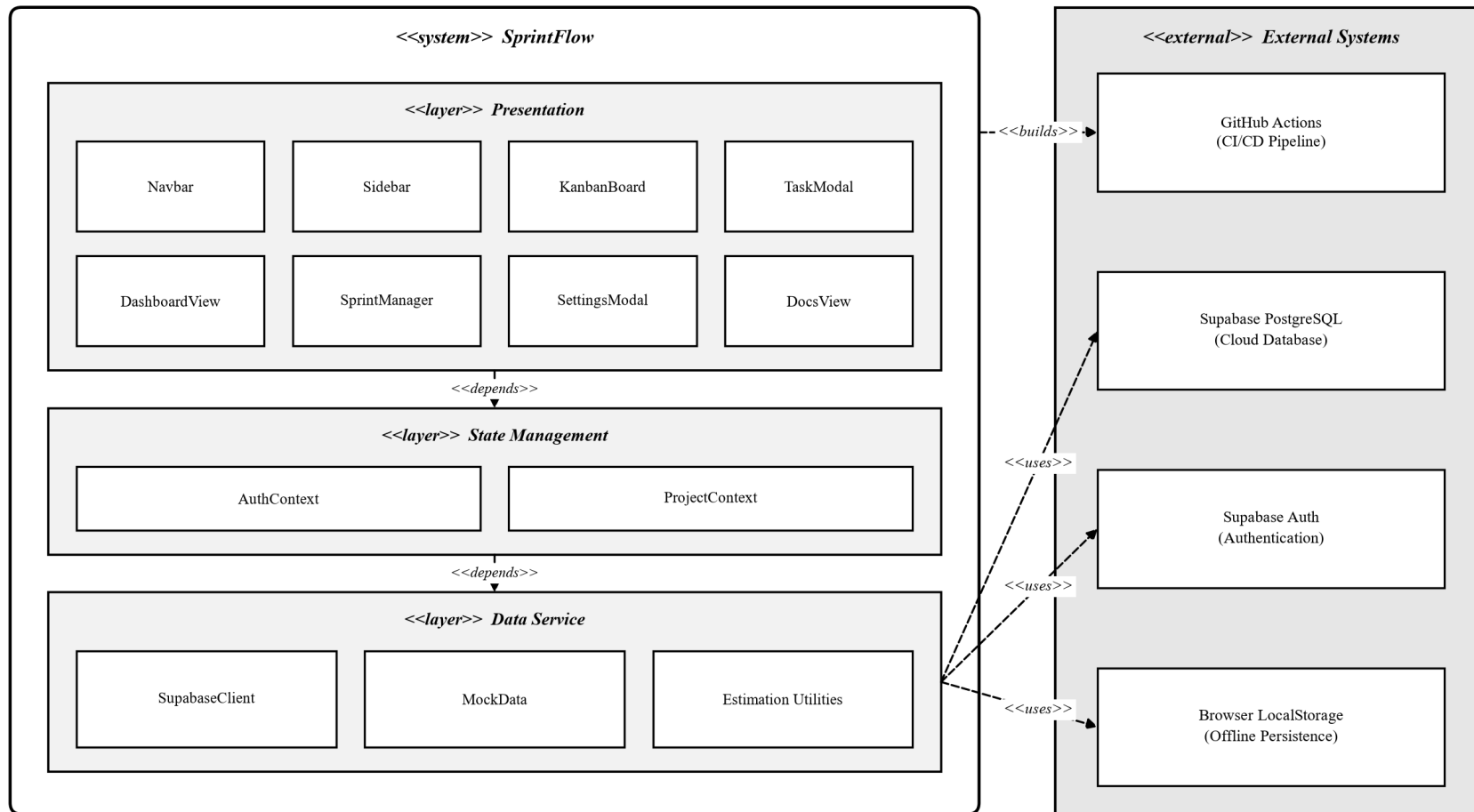


Figure 3.1: UML Component Diagram – SprintFlow System Architecture

The system is decomposed into the following primary components:

- **Presentation Layer:** Navbar, Sidebar, KanbanBoard, DashboardView, SprintManager, SettingsModal, TaskModal, DocsView
- **State Management Layer:** AuthContext, ProjectContext
- **Data Service Layer:** SupabaseClient (dual-mode: cloud + offline)
- **External Systems:** Supabase PostgreSQL, GitHub Actions CI/CD

3.4 Component Descriptions

Component	Module Path	Responsibility	Dependencies
App Shell	src/App.jsx	Root application container; manages view routing and global layout composition	AuthContext, ProjectContext, Navbar, Sidebar
Navbar	src/components/layout/Navbar.jsx	Top navigation bar with user profile display, view switching controls, and settings access	AuthContext
Sidebar	src/components/layout/Sidebar.jsx	Collapsible side navigation panel with project-level filters and quick-access links	ProjectContext
KanbanBoard	src/components/kanban/KanbanBoard.jsx	5-column Kanban workflow board (Backlog, To Do, In Progress, Code Review, Done) with HTML5 drag-and-drop state transitions	ProjectContext, TaskModal
TaskModal	src/components/kanban/TaskModal.jsx	Modal dialog for creating and editing task cards with Fibonacci story point assignment	ProjectContext
DashboardView	src/components/dashboard/DashboardView.jsx	Executive KPI dashboard with progress metrics, velocity calculations, workload distribution, and SVG burndown chart	ProjectContext
SprintManager	src/components/sprint/SprintManager.jsx	Sprint lifecycle management: creation, goal definition, capacity planning, and iteration tracking	ProjectContext
SettingsModal	src/components/settings/SettingsModal.jsx	Database configuration interface for Supabase connection parameters (Project URL, Anon Key)	SupabaseClient
DocsView	src/components/docs/DocsView.jsx	In-app documentation viewer for project guides and architecture references	None
AuthContext	src/context/AuthContext.jsx	React Context provider managing user authentication state, role profiles, and session persistence	SupabaseClient
ProjectContext	src/context/ProjectContext.jsx	React Context provider managing project data state: tasks, sprints, audit log, and reactive CRUD mutations	SupabaseClient, AuthContext
SupabaseClient	src/services/supabaseClient.js	Dual-mode data service: routes operations to Supabase PostgreSQL when configured, falls back to browser LocalStorage for zero-config offline operation	@supabase/supabase-js
MockData	src/services/mockData.js	Seed data generator providing pre-configured tasks, sprints, and user profiles for immediate evaluation	None
Estimation Utilities	src/utils/estimation.js	Fibonacci story point calculation engine (1, 2, 3, 5, 8, 13, 21) with sprint capacity algorithms	None
Constants	src/utils/constants.js	Application-wide configuration constants: workflow stages, priority levels, role definitions	None

3.5 Chosen Architecture Pattern and Rationale

Selected Pattern: Layered Component Architecture (React Component-Based SPA)

Rationale:

Criterion	Layered Component Architecture	Microservices	Monolithic MVC
Separation of Concerns	Strong (component encapsulation)	Very Strong (service isolation)	Moderate
Development Complexity	Low-Medium	High (service orchestration)	Low

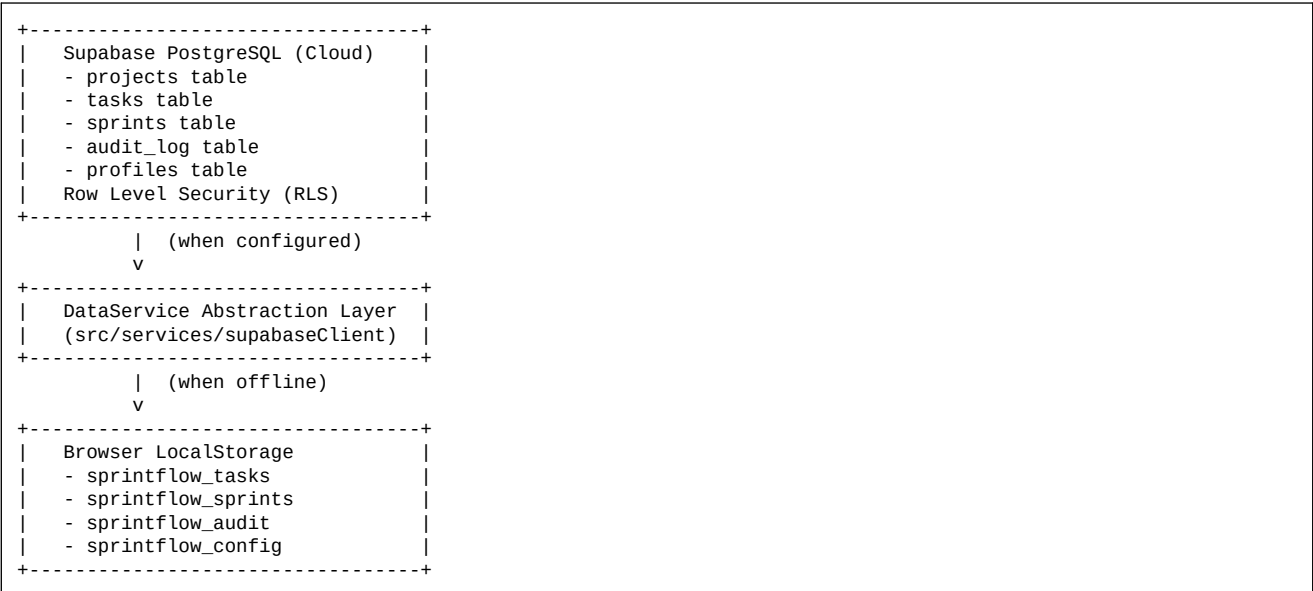
Criterion	Layered Component Architecture	Microservices	Monolithic MVC
Team Parallelism	High (independent component dev)	Very High	Low (merge conflicts)
Deployment Simplicity	Very High (static SPA bundle)	Low (multi-service orchestration)	Medium
Academic Evaluation Fit	Excellent (single npm run dev)	Poor (requires container infrastructure)	Good
Offline Capability	Native (client-side state)	Requires complex sync	Difficult

Decision: Layered Component Architecture was selected because it provides strong separation of concerns through React component encapsulation while maintaining deployment simplicity essential for academic evaluation. Microservices architecture was rejected as overly complex for the project scope, requiring container orchestration infrastructure that exceeds course requirements. Monolithic MVC was rejected due to limited support for team-parallel development and poor offline capability.

3.6 Technology Stack and Data Stores

Layer	Technology	Version	Justification
UI Framework	React	18.x	Industry-standard component library with mature ecosystem, hooks-based state management
Build Toolchain	Vite	5.x	Sub-second HMR, native ES modules, optimized production bundling
State Management	React Context API	Built-in	Sufficient for project scope; avoids external dependency overhead (Redux/MobX)
Cloud Database	Supabase PostgreSQL	Latest	Managed PostgreSQL with built-in RLS, real-time subscriptions, and REST API generation
Client SDK	@supabase/supabase-js	2.x	Official Supabase client with type-safe query builder and auth integration
Offline Storage	Browser LocalStorage	Web API	Zero-dependency persistent key-value store for offline fallback mode
Unit Testing	Vitest	2.x	Vite-native test runner with Jest-compatible API and fast execution
CI/CD	GitHub Actions	Latest	Integrated with repository; multi-job pipeline (build, test, lint, security audit)
Transport Security	TLS 1.3	Latest	Enforced by Supabase for all client-server communication

Data Store Architecture:



3.7 Risks and Mitigations

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
RISK-01	Supabase cloud service unavailability during evaluation	Medium	High	Dual-mode architecture with automatic LocalStorage fallback ensures 100% offline operation

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
RISK-02	Browser LocalStorage quota exceeded (5MB limit)	Low	Medium	Implement data pruning for audit logs exceeding 1000 entries; compress serialized state
RISK-03	Cross-browser drag-and-drop inconsistencies	Medium	Medium	Use HTML5 native DnD API with polyfill fallback; test across Chrome, Firefox, Edge
RISK-04	State synchronization conflicts between cloud and offline modes	Low	High	Last-write-wins conflict resolution with timestamp-based ordering
RISK-05	CI/CD pipeline failure blocking Pull Request merges	Low	Medium	Separate build and test jobs with independent failure isolation; manual merge override
RISK-06	Package dependency vulnerabilities in npm ecosystem	Medium	Medium	Automated <code>npm audit</code> in CI pipeline; regular dependency updates via Dependabot

3.8 Traceability to Requirements

The following table traces architectural components to the functional and non-functional requirements defined in SRS v1.0:

Requirement ID	Requirement Description	Architectural Component(s)
PMS-F-001	User authentication and profile roles	AuthContext, SupabaseClient
PMS-F-002	Create new task cards with metadata	TaskModal, ProjectContext
PMS-F-003	5-column Kanban board display	KanbanBoard
PMS-F-004	Drag-and-drop task stage transitions	KanbanBoard (HTML5 DnD API)
PMS-F-005	1-click task stage advancement	KanbanBoard
PMS-F-006	Fibonacci story point assignment	TaskModal, Estimation Utilities
PMS-F-007	Sprint creation and goal definition	SprintManager, ProjectContext
PMS-F-008	Sprint capacity calculation	SprintManager, Estimation Utilities
PMS-F-009	SVG burndown chart visualization	DashboardView
PMS-F-010	Executive KPI dashboard metrics	DashboardView
PMS-F-011	Task filtering and multi-criteria search	KanbanBoard, ProjectContext
PMS-F-012	Audit trail logging	ProjectContext, SupabaseClient
PMS-F-013	Supabase cloud database sync	SupabaseClient, SettingsModal
PMS-F-014	Offline LocalStorage persistence	SupabaseClient
PMS-F-015	Database settings configuration UI	SettingsModal
PMS-F-016	Task priority assignment	TaskModal
PMS-F-017	Workload balance per team member	DashboardView
PMS-F-018	In-app documentation viewer	DocsView
PMS-NF-001	Sub-50ms UI state transitions	KanbanBoard (optimized React rendering)
PMS-NF-002	99.9% availability target	Dual-mode SupabaseClient architecture
PMS-NF-003	Cross-browser compatibility	Vite build with ES module targets
PMS-NF-004	Data encryption in transit	TLS 1.3 (Supabase enforced)
PMS-NF-005	Support 100+ concurrent tasks	ProjectContext (efficient state updates)
PMS-NF-006	Recovery from network disconnection	SupabaseClient offline fallback

3.9 Security Architecture

Threat Modeling (STRIDE Analysis):

STRIDE Category	Threat Description	Countermeasure	Component
Spoofing	Unauthorized user impersonating a legitimate team member	Supabase Auth with email/password authentication; session token validation	AuthContext, SupabaseClient

STRIDE Category	Threat Description	Countermeasure	Component
Tampering	Malicious modification of task data or sprint records in transit	TLS 1.3 encryption for all Supabase API communication; input validation on all form fields	SupabaseClient
Repudiation	User denying actions performed on tasks or sprints	Immutable audit log with timestamp, user ID, action type, and affected entity recorded for every mutation	ProjectContext (audit_log)
Information Disclosure	Unauthorized access to project data belonging to other teams	PostgreSQL Row Level Security (RLS) policies restricting data access to authenticated project members only	database/schema.sql
Denial of Service	Excessive API requests overwhelming the Supabase backend	Client-side request debouncing; Supabase built-in rate limiting; offline fallback absorbs load during outages	SupabaseClient
Elevation of Privilege	Regular user attempting to access administrative functions	Role-based UI rendering; server-side RLS policies enforce role constraints independent of client-side checks	AuthContext, RLS policies

Security Controls Summary:

- **Authentication:** Supabase Auth with JWT token-based session management
- **Authorization:** PostgreSQL RLS policies enforcing row-level data isolation
- **Input Sanitization:** Client-side XSS prevention through React JSX auto-escaping and explicit input validation
- **Transport Security:** TLS 1.3 enforced on all Supabase client-server communication
- **Audit Logging:** Comprehensive mutation audit trail with non-repudiation guarantees
- **Dependency Security:** Automated npm audit in CI/CD pipeline scanning for known vulnerabilities

4. Design

4.1 Design Overview

SprintFlow is designed as a modular, component-based single-page application following the principle of separation of concerns at every layer:

- **Presentation Layer:** React functional components with hooks-based lifecycle management. Each UI module (Kanban, Dashboard, Sprint Manager) is independently encapsulated with its own rendering logic and event handlers.
- **State Management Layer:** Two React Context providers (AuthContext and ProjectContext) serve as centralized state containers, exposing reactive data and mutation functions to all consuming components via the useContext hook.
- **Data Access Layer:** A unified DataService abstraction in SupabaseClient routes all CRUD operations through a mode-detection mechanism that transparently selects between Supabase PostgreSQL (when configured) and browser LocalStorage (default offline mode).

This layered design enables each team member to develop and test their assigned components independently, with well-defined interfaces at layer boundaries.

4.2 UML Sequence Diagrams

4.2.1 Sequence Diagram 1: User Authentication and Task Creation Workflow

This sequence diagram models the end-to-end flow from user authentication through task creation on the Kanban board:

UML Sequence Diagram: User Authentication and Task Creation

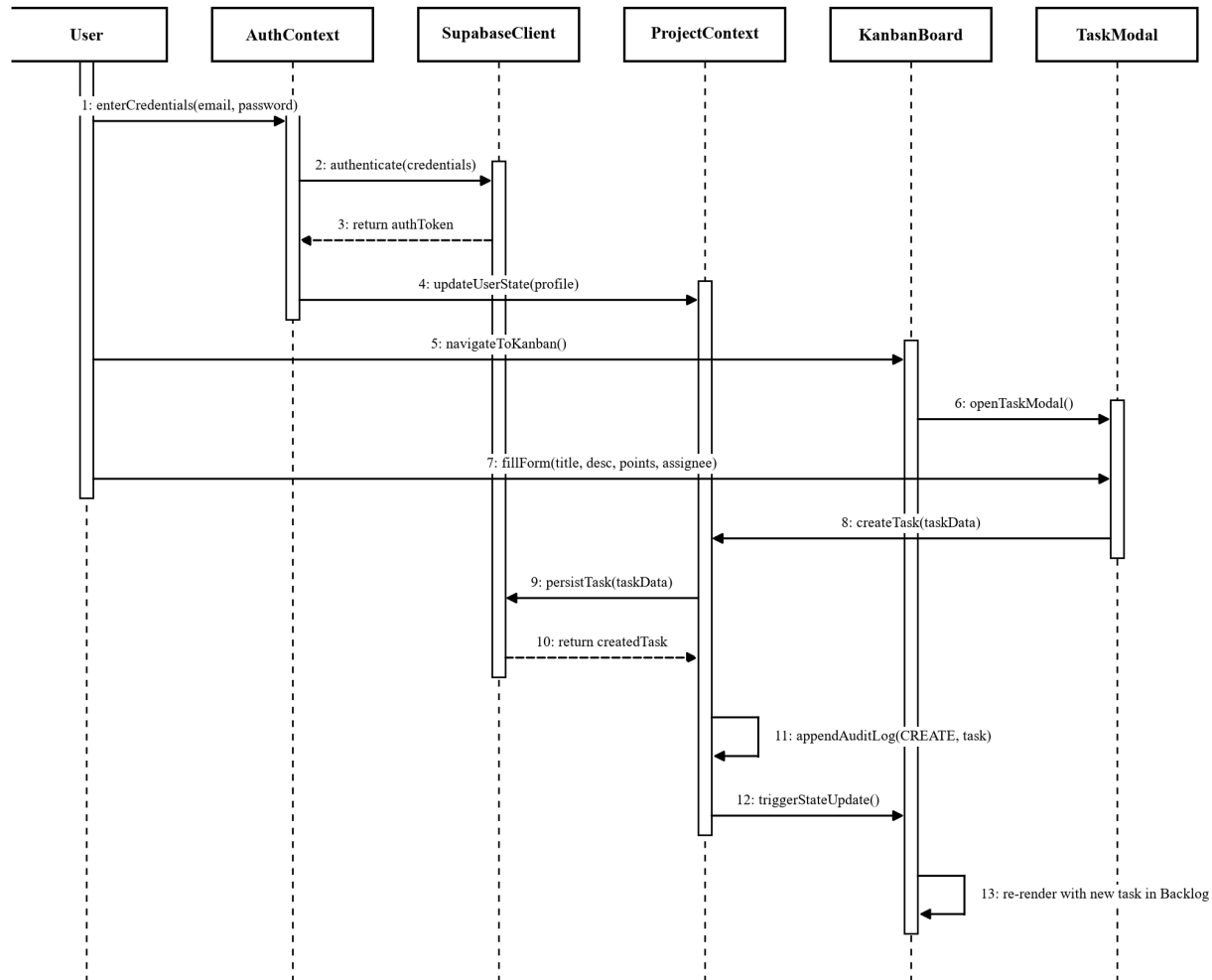


Figure 4.1: UML Sequence Diagram – Authentication and Task Creation

Flow Description:

1. User enters credentials in the login form
2. AuthContext dispatches authentication request to SupabaseClient
3. SupabaseClient routes to Supabase Auth API (cloud mode) or validates against LocalStorage credentials (offline mode)
4. On successful authentication, AuthContext updates user state and triggers UI re-render
5. User navigates to KanbanBoard and clicks "Add Task"
6. TaskModal renders with form fields (title, description, priority, story points, assignee)
7. User fills form and submits; TaskModal dispatches createTask to ProjectContext
8. ProjectContext forwards the mutation to SupabaseClient for persistence
9. SupabaseClient inserts the record and returns the created task
10. ProjectContext appends an audit log entry and triggers state update
11. KanbanBoard re-renders with the new task card in the Backlog column

4.2.2 Sequence Diagram 2: Sprint Estimation and Burndown Chart Generation

This sequence diagram models the sprint planning and burndown visualization workflow:

UML Sequence Diagram: Sprint Estimation and Burndown Generation

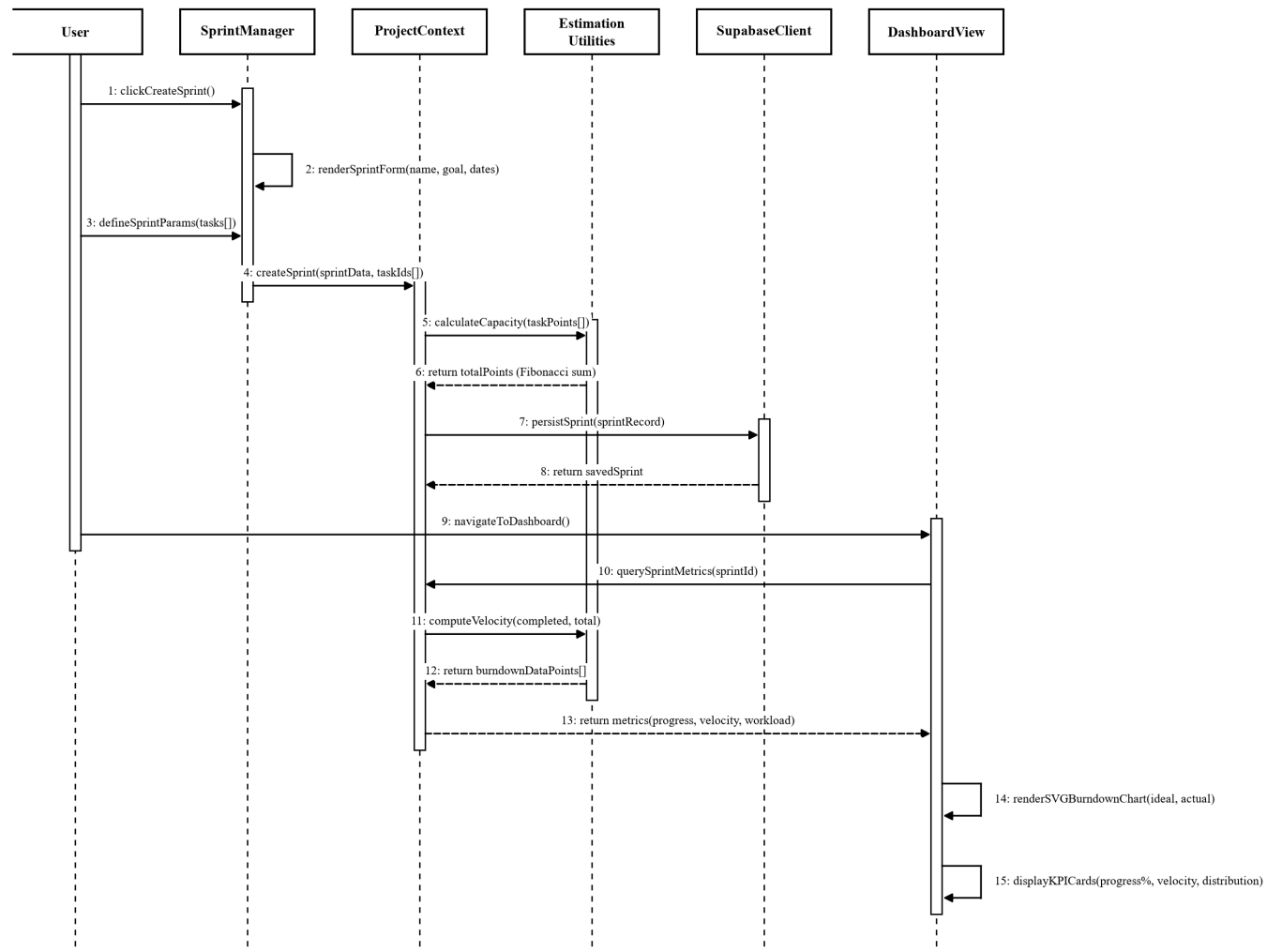


Figure 4.2: UML Sequence Diagram – Sprint Estimation and Burndown

Flow Description:

1. User navigates to SprintManager and clicks "Create Sprint"
2. SprintManager renders sprint creation form (name, goal, start date, end date)
3. User defines sprint parameters and assigns tasks from the backlog
4. SprintManager dispatches createSprint to ProjectContext with selected task IDs
5. ProjectContext calculates total sprint capacity using Estimation Utilities (Fibonacci sum)
6. SupabaseClient persists the sprint record and task-sprint associations
7. User navigates to DashboardView to monitor sprint progress
8. DashboardView queries ProjectContext for sprint metrics (total points, completed points, remaining points)
9. ProjectContext computes velocity and burndown data points
10. DashboardView renders the SVG burndown chart comparing ideal trajectory against actual burn rate
11. DashboardView displays KPI cards (progress percentage, velocity, workload distribution)

4.3 API Design

SprintFlow interfaces with Supabase PostgreSQL through the auto-generated REST API. The following documents the primary API contracts:

4.3.1 Tasks API

Endpoint	Method	Description
/rest/v1/tasks	GET	Retrieve all tasks for the authenticated user's project
/rest/v1/tasks	POST	Create a new task record
/rest/v1/tasks?id=eq.{taskId}	PATCH	Update an existing task (stage, priority, story points, assignee)
/rest/v1/tasks?id=eq.{taskId}	DELETE	Delete a task record

Request Schema (POST /rest/v1/tasks):

```
{
  "title": "string (required, max 200 chars)",
  "description": "string (optional, max 2000 chars)",
  "status": "string (enum: backlog | todo | in_progress | review | done)",
  "priority": "string (enum: low | medium | high | critical)",
  "story_points": "integer (enum: 1 | 2 | 3 | 5 | 8 | 13 | 21)",
  "assignee": "string (team member name)",
  "sprint_id": "uuid (optional, foreign key to sprints table)"
}
```

Response Schema (200 OK):

```
{
  "id": "uuid",
  "title": "string",
  "description": "string",
  "status": "string",
  "priority": "string",
  "story_points": "integer",
  "assignee": "string",
  "sprint_id": "uuid | null",
  "created_at": "ISO 8601 timestamp",
  "updated_at": "ISO 8601 timestamp"
}
```

Error Responses:

- 401 Unauthorized – Missing or invalid authentication token
- 403 Forbidden – RLS policy violation (user not authorized for this project)
- 422 Unprocessable Entity – Validation failure (missing required fields, invalid enum value)

4.3.2 Sprints API

Endpoint	Method	Description
/rest/v1/sprints	GET	Retrieve all sprints for the authenticated user's project
/rest/v1/sprints	POST	Create a new sprint with goal and date range
/rest/v1/sprints?id=eq.{sprintId}	PATCH	Update sprint status or goal

Request Schema (POST /rest/v1/sprints):

```
{
  "name": "string (required, max 100 chars)",
  "goal": "string (required, max 500 chars)",
  "start_date": "ISO 8601 date",
  "end_date": "ISO 8601 date",
  "status": "string (enum: planning | active | completed)"
}
```

Response Schema (200 OK):

```
{
  "id": "uuid",
  "name": "string",
  "goal": "string",
  "start_date": "date",
  "end_date": "date",
  "status": "string",
  "total_points": "integer (computed)",
  "completed_points": "integer (computed)",
  "created_at": "ISO 8601 timestamp"
}
```

4.4 Error Handling, Logging and Monitoring

Error Handling Strategy:

- All API calls are wrapped in try-catch blocks with standardized error response parsing
- Network failures trigger automatic fallback to LocalStorage mode with a user-visible notification
- Form validation errors are displayed inline with field-level error messages
- Unrecoverable errors are caught by a React Error Boundary component that renders a graceful fallback UI

Logging Strategy:

- All data mutations (create, update, delete) are recorded in the audit log with timestamp, user ID, action type, and affected entity
- No sensitive information (passwords, API keys) is written to any log
- Audit log entries are persisted through the same dual-mode data service (Supabase or LocalStorage)

Monitoring:

- CI/CD pipeline provides automated build health monitoring on every push
- Vitest test suite execution results are captured in GitHub Actions logs
- Client-side performance metrics (drag-and-drop latency, render time) are measurable through browser DevTools

4.5 UX Design

Design Principles:

- **Immediate Feedback:** All user actions (task creation, drag-and-drop, sprint operations) produce immediate visual feedback within 50ms
- **Progressive Disclosure:** Advanced features (database settings, sprint configuration) are accessible through modals and secondary navigation, keeping the primary interface clean
- **Consistency:** Uniform visual language across all views with consistent color coding for task priorities and workflow stages
- **Accessibility:** High-contrast dark theme with readable typography, keyboard-navigable interface elements, and semantic HTML structure

- **Responsive Layout:** Fluid grid layout adapting to viewport sizes from 1024px desktop to 1920px widescreen

Visual Design System:

- **Color Palette:** Dark glassmorphic theme with gradient accents for visual hierarchy
- **Typography:** System font stack optimized for readability across platforms
- **Spacing:** 8px grid system for consistent padding, margins, and component sizing
- **Interaction States:** Hover, focus, active, and disabled states defined for all interactive elements

4.6 Open Issues and Next Steps

Issue	Status	Next Step
Real-time collaborative editing (multiple users editing the same board)	Deferred	Evaluate Supabase Realtime subscriptions for live sync
Offline-to-online data reconciliation on reconnection	Partial	Implement queue-based sync with conflict detection
Role-based access control at the UI level (admin vs member permissions)	Implemented (basic)	Extend with granular permission matrix
Export sprint reports as PDF from the dashboard	Not started	Integrate client-side PDF generation library
Mobile-responsive Kanban board layout	Not started	Implement touch-friendly card interactions for tablet viewports

5. Appendices

5.1 Glossary

Term	Definition
Kanban	Visual workflow management method using cards and columns to represent work items and their status
Sprint	A time-boxed iteration (typically 1-4 weeks) in Agile methodology during which a set of work items is completed
Burndown Chart	A graphical representation of remaining work versus time in a sprint
Story Points	A unit of measure for expressing the overall effort required to implement a user story (Fibonacci scale)
Velocity	The amount of work (in story points) completed per sprint, used for future capacity planning
RLS	Row Level Security – PostgreSQL feature enabling fine-grained access control at the database row level
STRIDE	A threat modeling framework categorizing threats into six categories for systematic security analysis

5.2 References

Reference	Description
IEEE 42010:2011	Systems and Software Engineering – Architecture Description
OWASP Top 10 (2021)	Open Web Application Security Project – Top 10 Web Application Security Risks
NIST SP 800-160	Systems Security Engineering – Considerations for a Multidisciplinary Approach
React Documentation	Official React 18 documentation (react.dev)
Supabase Documentation	Official Supabase platform documentation (supabase.com/docs)
Vite Documentation	Official Vite 5 build tool documentation (vitejs.dev)

5.3 Tools

Tool	Purpose
Visual Studio Code	Primary development IDE
Git / GitHub	Version control and collaboration platform
GitHub Actions	CI/CD pipeline automation

Tool	Purpose
Vitest	Unit testing framework
Supabase Dashboard	Database management and RLS policy configuration
Browser DevTools	Performance profiling and debugging
Python (Matplotlib)	UML diagram generation for documentation